

1. INTRODUÇÃO A BANCO DE DADOS

Por que estudar Banco de Dados?

Vivemos em uma era definida pela informação. A cada segundo, bilhões de dados são gerados por transações comerciais, redes sociais, sensores industriais, sistemas de saúde e aplicações governamentais. Diante desse volume colossal, surge uma pergunta fundamental: como organizar, armazenar, recuperar e proteger esses dados de maneira eficiente e confiável? A resposta a essa pergunta é o objeto central desta disciplina: o **Banco de Dados**.

Para compreender a importância dos bancos de dados, é útil olhar para o passado. Antes da informatização, as organizações mantinham seus registros em fichas, livros contábeis e pastas arquivadas em estantes. Com o advento dos primeiros computadores, a solução imediata foi reproduzir essa lógica em arquivos digitais: cada departamento criava seus próprios arquivos, com seus próprios formatos e regras.

Esse modelo, conhecido como **processamento baseado em arquivos**, rapidamente revelou limitações graves. Havia redundância de dados — o endereço de um cliente, por exemplo, podia estar repetido em cinco arquivos diferentes. Havia inconsistência — ao atualizar o endereço em um arquivo, os demais permaneciam desatualizados. Havia dificuldade de integração — o setor financeiro não conseguia cruzar informações com o setor de estoque sem um esforço manual considerável. Além disso, a segurança e a concorrência de acesso eram problemas praticamente insolúveis.

Foi para superar essas limitações que surgiram os **Sistemas Gerenciadores de Banco de Dados (SGBD)**, softwares especializados que atuam como intermediários entre as aplicações e os dados armazenados.

Conceitos fundamentais

Antes de avançarmos, é essencial distinguir dois termos frequentemente confundidos:

- **Dado** é a representação bruta de um fato: um número, um nome, uma data. Isoladamente, ele possui pouco significado.
- **Informação** é o dado contextualizado, processado e organizado de modo a transmitir conhecimento útil para a tomada de decisão.

O **Banco de Dados** é, portanto, uma coleção organizada e integrada de dados, estruturada para atender às necessidades de informação de uma organização. Já o **SGBD** — como MySQL, PostgreSQL, Oracle ou SQL Server — é o software responsável por criar, manter e controlar o acesso a esse banco, garantindo integridade, segurança, concorrência e recuperação diante de falhas.

Um banco de dados não surge ao acaso. Antes de escrever qualquer linha de código, é preciso **modelar** a realidade que se deseja representar. A modelagem conceitual,

geralmente expressa por meio do **Modelo Entidade-Relacionamento (MER)**, permite identificar as entidades relevantes (Cliente, Produto, Pedido), seus atributos (nome, preço, data) e os relacionamentos entre elas. A partir desse modelo conceitual, deriva-se o **modelo lógico** — tipicamente o modelo relacional, proposto por Edgar Codd em 1970 —, no qual os dados são organizados em tabelas compostas por linhas e colunas. Por fim, chega-se ao **modelo físico**, que define como os dados serão efetivamente armazenados em disco, com índices, particionamentos e estruturas de acesso.

Essa progressão — do conceitual ao físico — é um dos pilares desta disciplina e será explorada em profundidade ao longo do curso.

A linguagem SQL

Para interagir com um banco de dados relacional, utilizamos a **SQL (Structured Query Language)**, uma linguagem declarativa padronizada que permite consultar, inserir, atualizar e excluir dados, além de definir e controlar a estrutura do banco. Dominar a SQL é uma competência indispensável para qualquer profissional de tecnologia, independentemente da área de atuação.

Contudo, o universo dos bancos de dados não se restringe ao modelo relacional. Nas últimas décadas, surgiram os chamados **bancos NoSQL** — orientados a documentos, grafos, colunas ou pares chave-valor —, projetados para cenários de alta escalabilidade, dados semiestruturados e arquiteturas distribuídas. Conhecer esses paradigmas amplia a capacidade do profissional de escolher a solução mais adequada para cada problema.

2. OS PRIMÓRDIOS DOS BANCOS DE DADOS

No final da década de 1950 e ao longo dos anos 1960, os computadores deixaram de ser máquinas exclusivas de cálculos militares e científicos para começar a atuar no mundo dos negócios. Empresas de grande porte — bancos, seguradoras, indústrias e órgãos governamentais — passaram a informatizar suas operações: folha de pagamento, controle de estoque, faturamento e cadastro de clientes.

O desafio, porém, era imediato: onde e como guardar essa quantidade crescente de registros? A primeira resposta foi a mais intuitiva — **arquivos sequenciais em fita magnética**. Os dados eram gravados em fitas, lidos do início ao fim, e qualquer consulta exigia percorrer o arquivo inteiro até encontrar o registro desejado. Imagine procurar um único cliente em uma fita com dois milhões de registros: era necessário rebobinar e avançar sequencialmente até localizá-lo.

Com o surgimento dos **discos magnéticos de acesso direto**, no início dos anos 1960, tornou-se possível saltar diretamente a uma posição de armazenamento. Isso permitiu a criação dos primeiros **sistemas de gerenciamento de arquivos**, nos quais cada aplicação possuía seus próprios arquivos, com seus próprios formatos e programas de acesso.

Embora representasse um avanço em relação às fitas, o modelo baseado em arquivos logo revelou problemas sérios que motivaram a criação dos bancos de dados propriamente ditos:

- **Redundância e inconsistência:** o mesmo dado (por exemplo, o endereço de um funcionário) era repetido em diversos arquivos de setores distintos. Uma atualização em um arquivo não se propagava aos demais, gerando versões conflitantes da informação.
- **Dependência entre dados e programas:** a estrutura física do arquivo estava embutida no código da aplicação. Alterar o formato de um campo exigia reescrever programas.
- **Dificuldade de acesso integrado:** responder a uma pergunta que envolvesse dados de dois ou mais arquivos distintos demandava programação complexa e demorada.
- **Problemas de concorrência e atomicidade:** dois usuários acessando o mesmo arquivo simultaneamente podiam corromper dados ou gerar resultados inconsistentes.

Ficou claro que era necessário um **sistema centralizado**, capaz de gerenciar os dados de forma independente das aplicações que os utilizavam.

2.1 O MODELO HIERÁRQUICO

A resposta pioneira veio em **1966**, quando a IBM, em parceria com a Rockwell International e a Caterpillar, desenvolveu o **IMS (Information Management System)** para apoiar o programa espacial Apollo. O IMS implementava o chamado **modelo hierárquico**, no qual os dados eram organizados em uma estrutura de **árvore**.

Nessa estrutura, cada registro (chamado de segmento) possuía um único "pai" e podia ter múltiplos "filhos". Pense, por exemplo, em uma hierarquia: no topo está o registro da Empresa; abaixo dele, os Departamentos; abaixo de cada Departamento, os Funcionários; e abaixo de cada Funcionário, seus Dependentes. A navegação pelos dados seguia caminhos predefinidos de cima para baixo.

O modelo hierárquico trouxe vantagens importantes: reduziu a redundância, centralizou o controle dos dados e ofereceu desempenho elevado para consultas que seguiam exatamente a hierarquia prevista. O IMS permanece em uso até hoje em grandes instituições financeiras, processando milhões de transações diárias.

2.2 O MODELO EM REDE

No final da década de 1960, a **CODASYL** (Conference on Data Systems Languages), um consórcio que reunia especialistas da indústria e do governo norte-americano, propôs o **modelo em rede**, também conhecido como **modelo CODASYL**. O engenheiro **Charles Bachman**, na General Electric, foi um dos principais idealizadores dessa abordagem, tendo desenvolvido o sistema **IDS (Integrated Data Store)**, precursor do conceito.

No modelo em rede, os registros podiam ter **múltiplos pais e múltiplos filhos**, formando uma estrutura de **grafo** em vez de árvore. Isso permitia representar relacionamentos mais complexos: um funcionário podia pertencer a mais de um departamento, e um produto podia estar associado a vários fornecedores. A navegação era feita por meio de **ponteiros físicos** (links) que conectavam os registros entre si.

O modelo em rede oferece maior flexibilidade que o hierárquico e melhor desempenho em consultas que percorriam caminhos múltiplos. Por essas contribuições, Charles Bachman recebeu o **Prêmio Turing** em 1973, a mais alta honraria da Ciência da Computação.

Limitações

Apesar dos avanços, tanto o modelo hierárquico quanto o modelo em rede compartilhavam limitações significativas:

- **Complexidade de navegação:** para obter uma informação, o programador precisa conhecer a estrutura física do banco e percorrer manualmente os ponteiros, registro por registro. Uma consulta simples podia exigir dezenas de comandos de navegação.
- **Alto acoplamento entre lógica e estrutura:** se a hierarquia ou os links fossem alterados, todos os programas que acessavam aqueles caminhos precisavam ser reescritos.
- **Dificuldade de consultas ad hoc:** perguntas não previstas no projeto original eram extremamente difíceis de formular. Não existia uma linguagem declarativa na qual o usuário simplesmente declarasse *o que* desejava; era preciso especificar *como* navegar até a resposta.
- **Rigidez na modelagem:** mudanças nos requisitos de negócio frequentemente exigiam reestruturação física dos dados, com impacto em todas as aplicações.

Ao final dos anos 1960, a comunidade computacional reconhecia que era preciso separar definitivamente a **representação lógica** dos dados da sua **implementação física**. Era necessário que o usuário pudesse solicitar informações sem saber como os dados estavam armazenados em disco, sem percorrer ponteiros e sem conhecer hierarquias. Foi exatamente nesse contexto de insatisfação e busca por simplicidade que, em **1970, Edgar Frank Codd**, pesquisador da IBM, publicou o artigo "*A Relational Model of Data for Large Shared Data Banks*". Nele, propôs que os dados fossem tratados como **conjuntos matemáticos de tuplas**, organizados em relações (tabelas), acessados por operações da álgebra relacional — e não por navegação em ponteiros.

Essa ideia, que parecia puramente teórica no momento de sua publicação, transformaria radicalmente a forma como o mundo armazena e consulta informação, inaugurando a era dos bancos de dados relacionais que estudaremos em profundidade nesta disciplina.

3. O QUE É DADO?

A palavra **dado** vem do latim *datum*, que significa "aquilo que foi dado", "o que é concedido". Em sua acepção mais ampla, dado é qualquer **representação bruta de um fato, evento, objeto ou fenômeno**, expressa por meio de símbolos, números, caracteres, imagens ou sons, que ainda não foi interpretada, contextualizada ou organizada para transmitir um significado específico. Em outras palavras, o dado é a matéria-prima da informação. Isoladamente, ele não responde a perguntas, não orienta decisões e não gera compreensão. Ele simplesmente é.

Dado, Informação e Conhecimento

- **Dado:** "38,5". Trata-se apenas de um número, sem contexto. Pode ser uma temperatura, uma nota, um código.
- **Informação:** "A temperatura corporal do paciente João é 38,5 °C". Agora o dado foi contextualizado, associado a uma entidade e a uma unidade de medida. Passa a ter significado.
- **Conhecimento:** "Temperatura de 38,5 °C indica febre; é necessário investigar possível infecção". Aqui, a informação é interpretada à luz da experiência e de regras, permitindo ação e julgamento.

Perceba que o dado é o **primeiro degrau** dessa escada. Sem ele, não há informação; sem informação, não há conhecimento. No contexto dos bancos de dados, nosso interesse recai fundamentalmente sobre os dois primeiros níveis: armazenar dados de forma estruturada para que possam ser transformados em informação útil.

3.1 TIPOS E FORMATOS DE DADOS

Os dados podem se manifestar de diversas formas:

- **Numéricos:** inteiros, decimais, valores monetários (ex.: quantidade em estoque = 120).
- **Textuais:** nomes, descrições, endereços (ex.: "Rua das Acácias, 45").
- **Temporais:** datas, horários, carimbos de tempo (ex.: 2025-03-15 14:30).
- **Lógicos/Booleanos:** verdadeiro ou falso (ex.: cliente ativo = verdadeiro).
- **Multimídia:** imagens, áudios, vídeos, documentos em PDF.

Independentemente do formato, todo dado precisa ser **representado** de alguma maneira para ser armazenado e processado computacionalmente. No nível mais baixo, essa representação é binária — sequências de zeros e uns. Nos níveis mais altos, com os quais trabalhamos em SQL, utilizamos tipos como **INTEGER**, **VARCHAR**, **DATE**, **BOOLEAN** e **BLOB**.

3.2 DADO NO CONTEXTO DE BANCO DE DADOS

Quando projetamos um banco de dados, cada dado é associado a um **atributo** (ou campo) dentro de uma entidade. Por exemplo, na entidade *Cliente*, podemos ter os atributos *Nome*, *CPF*, *Data de Nascimento* e *E-mail*. Cada instância concreta — "Maria Silva", "123.456.789-00", "1990-07-22", "maria@email.com" — constitui um conjunto de **valores**, ou seja, dados registrados. É fundamental compreender que o dado, por si só, não possui utilidade. Ele só ganha relevância quando inserido em uma **estrutura** (o esquema do banco), associado a uma **semântica** (o que aquele campo representa) e vinculado a uma **finalidade** (para que será consultado).

Entender o que é dado — e, sobretudo, o que ele *ainda não é* — é o primeiro passo para projetar bancos de dados consistentes, evitar redundâncias e garantir que a informação extraída seja confiável. Ao longo desta disciplina, veremos como modelar, armazenar, consultar e proteger esses dados, transformando registros brutos em conhecimento acionável para pessoas e organizações.